

Internet stvari i servisa

Projekat 1

Komparativna analiza sinhronih komunikacionih paradigmi u IoT mikroservisnim sistemima

Cilj je istražiti performanse i opravdanost korišćenja tri dominantna sinhrona modela komunikacije (**REST**, **gRPC** i **GraphQL**) u specifičnom kontekstu Interneta stvari (IoT). Fokus je na razumevanju kako izbor protokola utiče na latenciju, mrežni saobraćaj i procesorske resurse u *edge-cloud* kontinuumu.

Student bira javno dostupan IoT dataset (npr. *Smart City*, *Environmental Monitoring*, *Smart Home*, *Energy Grid*, *Smart Mobility*,...). Dataset mora biti vremenski serijalizovan (posedovati timestamp) i sadržati bar 3-5 različitih senzorskih podataka (npr. temperatura, vlaga, CO2, napon, GPS koordinate).

- <https://data.world/datasets/iot>
- <https://ieee-dataport.org/topic-tags/iot>
- <https://www.kaggle.com/search?q=IoT>
- <https://hub.packtpub.com/25-datasets-deep-learning-iot/>
- ...

Zadatak:

1. Dizajnirati **PostgreSQL** bazu podataka optimizovanu za IoT (indeksiranje po vremenu i ID-u uređaja).
2. Implementirati tri odvojena mikroservisa (u bar dve različite tehnologije, npr. ASP.NET Core, Node.js, Python/FastAPI, Java/Spring Boot) koji pristupaju istoj bazi:
 - **REST servis**: Standardni endpointi sa JSON formatom i OpenAPI (Swagger) dokumentacijom.
 - **gRPC servis**: Binarna komunikacija putem .proto definicija (Protobuf).
 - **GraphQL servis**: Omogućiti klijentu selektovanje specifičnih polja (izbegavanje over-fetching-a).
3. Svaki student mora validirati svoj sistem kroz tri specifična IoT scenarija. Evaluacija se vrši nad sistemom koji je u potpunosti kontejnerizovan pomoću **Docker Compose-a**.
 - **Scenario A (High-Frequency Ingestion)**: Simulacija uređaja koji šalje podatke u kratkim intervalima. Fokus na brzini upisa i overhead-u protokola.
 - **Scenario B (Selective Monitoring)**: Scenario gde klijent (npr. mobilna app) sa lošom vezom traži samo 2 od 10 dostupnih senzorskih vrednosti.
 - **Scenario C (Heavy Querying)**: Složeni upiti nad velikim opsegom istorijskih podataka (agregacije).
4. Merenje i evaluacija performansi
 - a) Koristiti alat **k6** i napisati k6 skriptu koja simulira različita opterećenja (10, 100, 500 virtuelnih korisnika) i generisati **metrike**: prosečna latencija (`http_req_duration`), p95 latencija i broj uspešnih zahteva u sekundi (RPS).
 - b) Koristiti **Postman** (Console tab) (eventualno **Wireshark**) i za identičan set podataka izmeriti veličinu odgovora (u bajtovima) za sva tri protokola. **Fokus**: Uporediti JSON (REST/GraphQL) sa binarnim Protobuf formatom (gRPC).
 - c) Koristiti **docker stats** (opciono za napredne: Prometheus + Grafana) i pratiti zauzeće CPU i RAM-a za svaki kontejner tokom trajanja k6 load testa. **Analiza**: Koliko "košta" serijalizacija/deserijalizacija podataka na procesorskom nivou?

Izvorni kod projekta postaviti na GitHub, kao i dokument sa opisom implementiranih REST API (OpenAPI), gRPC (proto) i GraphQL servisa i rezultata analize i evaluacije performansi servisa.